

Finding Minimum Volume Enclosing Ellipses for Tighter Upper Bounds on Error Probabilities for Controllers

Asher Arya

Affiliation: University of British Columbia, Vancouver, Canada

ORCID: 0009-0008-0774-7613

e-mail: asher.arya@gmail.com

Keywords: autonomous controller, monte carlo, state partitioning

I. Introduction

A. Problem Domain and Statement

Computer based control is ubiquitous in modern devices ranging from toys that accept voice commands ([16]) through autonomous vehicles and life-critical medical equipment. When controllers are used in applications where failures can be catastrophic, such as autonomous vehicles or medical equipment, we would like to verify, preferably using mathematically rigorous methods, that the system never fails. In practice, this is an unrealistic ideal because of the limited scalability of existing formal techniques. Furthermore we can't realistically require that the environment, the context in which the device is operating, never imposes some extremely adversarial circumstances. For example, we might wish to require that a self driving car will *never* collide with a pedestrian. But, what if a pedestrian, dressed entirely in black, sprints out from behind a hedge in front of a vehicle at night in heavy rain? One can argue that no car can have a short enough stopping distance to avoid a collision under such circumstances. One way to address such problems would be to write a specification that has exceptions for highly adversarial actions by the environment. However, such a specification would have an extremely long list of exceptions, and that list would almost certainly be incomplete. An alternative is to observe that extremely adversarial actions by the environment are very rare. We then turn to the problem of verifying that failures of our system such as colliding with a pedestrian, are very rare. In other words we want to verify a very high Mean-Time Between Failures (MTBF).

The dominant approach to validating control systems in current practice is to use extensive simulation. Let's say that we require a MTBF of one million hours (and this may not be high enough for a self-driving car). To establish this with Monte-Carlo simulations, we need to simulate well-over one million hours of vehicle operation. A key question is "Can one establish an MTBF of H hours with less effort than simulating some multiple of H hours

of operation?”. Another way to phrase this is in terms of “events” per hour. Can one establish a failure probability of at most ϵ per event while simulating fewer than $\Omega(1/\epsilon)$ events.

A common approach to this question is to use “subset sampling” ([8]), where some number of Monte-Carlo simulations are performed using the outcomes from this subset of rare trajectories as starting points. The fundamental problem with subset sampling is choosing a good criteria for “rare”. If our system operates in a continuous state space, we can find an arbitrarily large number of criteria for “rare”—just construct a ball around any point in the space—if the ball is small enough, the set of trajectories reaching that ball will be small enough to qualify as rare. In practice, designers try to define a criterion for rare that intuitively coincides with “likely to fail”, but we are not aware of any prior work that verifies such intuitive reasoning.

Recently, Justin Reiher, a Ph.D. student working with Ian Mitchell and Mark Greenstreet has proposed a method for establishing provable, upper bounds on failure probabilities using what he calls *Quasi-Markov Chains (QMC)*. Generally, Markov Chains are used to assign probabilities with discrete state changes ([5]), but QMCs use over-approximations of the transition probabilities which can sum to greater than 1. The method requires that the human performing the verification partitions the state space into regions. Each of these regions is a state of the QMC. One can compute provable upper-bounds for the elements of the QMC transition matrix by simulation if there are enough samples exhibiting the transition, or by using optimization methods for regions where few or even no trajectories are encountered using simulations. A challenge in this approach is that the human performing the verification must find a suitable partition of the state space. If they propose a poor partitioning, the method is still sound – it will just fail to verify the desired MTBF. To get useful results, we need a good partition. Presently, this partitioning remains an art.

B. Thesis Statement

As described above, Justin Reiher is developing analytical methods based on Quasi Markov Chains that provide a rigorous proof of lower bounds for mean-time between failure. In practice, a critical part of this approach is partitioning the state space into regions and establishing upper-bounds for the transition probabilities between these regions. Finding a good partitioning is a challenge in this approach.

In this thesis, I show that large-scale simulations can be used to generate visualizations that allow the human control engineer to both gain insights into the state space, and “debug” their proposed partitions. Data parallel computing on GPUs allows us to generate several orders of magnitude more samples than is practical using the Python code of published

control-examples. While simulations are not sufficient to establish very large MTBFs, we show that they are helpful for understanding the issues of state space partitioning for QMCs.

We demonstrate this approach with two examples. The first is a simple PD controller to move between two points in the plane without violating a safety-constraint expressed as a linear inequality. The second, is the classic “cart-pole” problem: controlling the motion of a cart along a track to bring it to rest at a designated destination while keeping an inverted pendulum in a near vertical orientation. Both models are discrete time with continuous state and include an additive stochastic disturbance at each step.

The remainder of this thesis is organized as follows:

Chapter 2 describes Reiher’s QMC technique and presents a survey of related work.

Chapter 3 describes the simple PD controller and the use of CUDA-based massive simulations to visualize the partition used to define the QMC. This also allows us to generate sampling-based estimates of the per-time-step failure probabilities for high-noise variations of the model. **We can show failure probabilities in the range of 10^{-7} down to 10^{-8} .** While these failure probabilities are not low enough to establish the desired target, they provide a validation test of the QMC, and let us quantify the impacts of the upper-bound approximations used by the QMC method.

Chapter 4 describes the cart-pole controller. While finding a meaningful partitioning of the state-space for the PD controller was straightforward, the cart-pole proved to be more challenging. Our work here focused on finding good partitioning methods.

Chapter 5 summarizes the results and describes opportunities for further research.

II. Background and Related Work

A. Quasi-Markov Chains

Quasi-Markov Chains (QMCs) [Reiher-thesis] extend Markov chains by allowing each entry of the transition matrix to be an upper bound (over-approximation), rather than an exact probability mass. This means that the sum of probabilities of transitioning to a different state can be greater than 1. The QMCs are recomputed for each pair of adjacent time steps t to $t + 1$. This is necessary to avoid violating the stationary assumption since the region definitions may evolve over time as the state space changes.

The procedure for building a useful QMC for a discrete-time continuous-state controller comprises three key steps [Reiher-thesis]. The first is the state-space partitioning, where the continuous state space is divided into discrete regions used for the QMTM. The second step involves using sampling and sample inflation to establish basic bounds on the transition probabilities, followed by optimization techniques to generate refined probabilities. The third step is build and use the QMTM to determine failure probabilities for each time step.

The way regions are defined should reflect both the state-space probabilities, and the system's expected evolution. Reiher [Reiher-thesis] defines regions by using norms to approximate probability level curves of the state distribution. He begins by measuring state deviations using either a Lyapunov function (if available) or the L_2 -norm distance from the expected trajectory. With this norm, he selects a percentage (typically 90%) of Monte Carlo samples closest to the nominal point to fit a convex geometric shape, which is typically an ellipsoid, since they can capture skew in the state distribution. This shape induces its own norm $\|\cdot\|_{M(t)}$ which is used to define approximate probability level curves for the system state. He defines a series of nested regions based on this norm, with smaller regions containing states more likely to be visited. He defined a maximum boundary $r_t^{\max}$ as the largest radius such that all states within it are guaranteed to be safe. This creates a safety set $S_t = \{x_t \text{ s.t. } \|x_t\|_{M(t)} \leq r_t^{\max}\}$. The primary innovation that captures the system's expected evolution is that Reiher defines regions not as these probability level sets, but as the set of states that will transition from probability level set to another across one time step. Specifically, he defines regions as:

$$R_t^i = \{x_t | r_{t+1}^i \leq \|f(x_t)\|_{M(t+1)} \leq r_{t+1}^{i+1}\} \cap S_t$$

where $f(x_t)$ is the expected next state. This means R_t^i consists of states that should transition to states between the i th and $(i + 1)$ th probability levels at the next time step. The final region, $R_t^{\max}$ captures all remaining states, including failure states, accounting for over-approximation of failure probabilities.

When dealing with sampling-based probability estimations, Reiher treats transition probabilities as Bernoulli random variables [11]. This approach considers the probability of making a transition from region R_t^i to R_{t+1}^j and treats all n_i samples as independent and identically distributed (IID) trials that have probability p_{ij} of ending in R_{t+1}^j . For large sample sizes, the Bernoulli distribution can be approximated as a Gaussian distribution N_j . With this approximation, we can establish upper bounds on transition probabilities with confidence γ by calculating how confident we are that N_j is greater than or equal to n_j , expressed as $P(N_j \geq n_j) \leq \gamma$. While this statistical approach provides reasonable approximations, the accuracy is limited to $O(1/N)$, where N is the number of Monte Carlo samples.

To achieve better approximations, optimization techniques can be employed to establish tighter bounds on transition probabilities. The optimization approach involves solving for the maximum probability of transitioning from one region to another by finding the state within the source region that maximizes the probability of reaching the destination region [11]. This involves solving a constrained optimization problem:

$$\max_{x_t \in R_t^i} \min_{x_{t+1}} \|x_{t+1}\|_{m(x_{t+1}|x_t)} \text{ s.t. } x_{t+1} \in R_{t+1}^j$$

The solution to this optimization problem provides an upper bound on the transition probability that can be significantly smaller than what could be established through sampling alone, especially for rare transitions. Since these optimized probabilities could be greater than the sampled probabilities, the minimum of the two is taken as the final upper bound.

Once the QMTM is constructed, failure probabilities can be calculated by analyzing how the probabilities distribute across regions over time [11]. The per-time-step failure probability is then the probability of entering the failure region at that time step, and the overall system failure probability is the sum of these per-time-step failure probabilities up to the time horizon of interest.

B. Related Work

Uncertainty in control systems can be categorized into two main types: aleatoric and epistemic [11]. Aleatoric uncertainty refers to the inherent randomness of the system, such as measurement errors, that cannot be reduced. Epistemic uncertainty refers to the systemic unmodeled dynamics and non-deterministic sections of the system that could potentially be reduced with more information or better modeling. Robustness, then, refers to the ability to respond to disturbances from these uncertainties. In general, robustness to all sources of disturbance cannot be guaranteed, and failure is possible. Therefore, a measure of failure likelihood or an approximation of failure is desired. An upper bound on the probability of failure would be the best metric for this, which is generally uncommon

in the literature review I performed. Most evaluations of failure find an approximation or estimation of the likelihood of failure.

When noise is Gaussian and the system is linear, the state can be estimated using techniques such as Kalman filtering [9]. For nonlinear systems with Gaussian noise, the Extended Kalman Filter (EKF) can be employed, which finds the best unbiased estimator of linearized dynamics. However, the EKF’s performance may degrade for highly nonlinear systems. The Unscented Kalman Filter [15] (UKF) addresses this by deliberately sampling the nonlinear system to obtain better estimates.

Various approaches exist for analyzing the safety of control systems. Level set methods compute reachable sets to determine whether a system can enter unsafe states [6, 10]. However, these methods often do not scale well to higher-dimensional systems and can be overly pessimistic in their over-approximations [11]. This limitation can sometimes be alleviated by breaking the system into smaller subsystems and analyzing them separately [6], or by reproducing kernel Hilbert space to handle million-dimensional systems [14], but calculating failure probability was not mentioned. Model checking techniques similar to Reiher’s method [11] treat transition probabilities as IID trials of a Bernoulli random variable approximated by a Gaussian in order to estimate the probability of failure [1]. However, while they generate statistical estimates that may be too optimistic, Reiher’s method uses optimization and sample inflation to find provable upper bounds.

Rare event sampling techniques are the most widely used estimators of failure probabilities in systems where failure is extremely unlikely, such as autonomous vehicles or medical devices [11]. The two primary approaches are subset simulation and importance sampling. Subset simulation [4] begins with a region of interest, which is typically the failure region, and defines a surrounding region p_0 , which is typically chosen to be 10% of the probability mass. A round of N Monte Carlo simulations is performed, and the region that surrounds the output of those simulations, conditioned on the samples being inside p_0 , is determined. This process repeats K times until the number of samples in the failure region exceeds $p_0 \times N$, in which case the loop terminates and the failure probability is computed. Note that the repeated iteration generates shrinking probability levels, with each new inner layer having a probability of p_0^k . The failure probability, then, is estimated as $\frac{p_0^{K-1} \times N_f}{N}$, where N_f is the number of samples in the failure region when the algorithm terminates. Importance sampling [2, 3] is another approach that requires a stricter distribution over failure regions, and it is often unclear what this distribution should be. Both importance sampling and subset sampling and its variants suffer from a major limitation. A function that scores samples with respect to their likelihood of failure, similar to a Lyapunov function, is required in order to generate regions of interest. With an incomplete distribution, or purely by chance,

it is entirely possible that a very important failure region that would significantly alter the failure probability is completely left out. It is due to this that rare-event sampling methods do not provide provable upper bounds.

Point estimate methods using Signal Temporal Logic (STL) [12, 13] are similar to subset sampling, but can suffer from particle collapse, where the algorithm attributes the failure probability to a single trajectory that happened to sample the worst noise.

Recent work by Fan and her students [7] on the cart-pole example is particularly relevant to our work. They use machine learning to generate both a controller and a Lyapunov function for the system, which can inform better state space partitioning strategies. Their Lyapunov function serves as a valuable measure of system stability and informs our region definition approach for the cart-pole controller.

III. The Simple PD Controller

A. Introduction

The first attempt at verification was with a two-dimensional state using a Proportional-Derivative (PD) controller. The state is the position and velocity along a line, represented with x and $\dot{x}$, respectively. A PD controller consists of 2 parameters, and is represented by the following equation:

$$u(t) = K_p e(t) + K_d \frac{d}{dt} e(t)$$

Where $u(t)$'s unit is typically m/s^2 , and $e(t) = \hat{x} - x$, where $\hat{x}$ is the setpoint. The first parameter, proportional gain (K_p), multiplies directly with the error and acts to counteract it. A K_p that is too high could result in overshooting and instability, while a K_p that is too low results in slow convergence, or being drowned out by stochastic noise (if the system uses this). The second parameter, derivative gain (K_d), multiplies with the derivative of the error to act as a dampener, allowing faster convergence. A PD controller is a reduced version of a PID controller, which incorporates the integral gain term $K_i \int_0^t e(\tau) d\tau$. K_i multiplies with the cumulative total error to settle smoothly at the setpoint. Note that if the nominal point is $(0,0)$, the PD equation degenerates to $u(t) = K_p x_t + K_d \dot{x}_d$. We use a setpoint of $(0,0)$ in this paper, and thus the degenerate equation for the PD controller.

This simple linear deterministic system is made slightly more complex with the application of stochastic noise to the state. Without it, however, the over-approximation of failure would be of little use, since the problem would be entirely straightforward. Even with this addition, however, if applied correctly, the problem can remain deterministic. This determinism is useful, as an over-approximation of the MTBF can be compared with the deterministic MTBF. Gaussian noise was chosen due to it keeping the system deterministic and was added independently to the two components of the state, x and $\dot{x}$.

The choice of region shapes and method of partitioning will directly influence the precision and accuracy of our over-approximation. For example, if the state space is partitioned into hyper-rectangles of varying sizes and locations, the over-approximation may be too large due to under representation of rare regions or over-representation of failure, or may require specific knowledge and/or testing of the model, which runs contrary to the purpose of this paper's method. If n-spheres are used, then, they may capture regions better, but intersections of spheres and skew may become an issue. Ellipsoids, then, can better capture skew in various orientations in the state space. Finding regions currently remains controller-specific, with strong reason to be automated. In the case of the simple PD controller, concentric ellipses around the setpoint capture both skew and the Gaussian probability levels well, so they will be used. These ellipses will contain a set percentage ($P\%$) of the

samples outside of any inner ellipses.

I address the challenge of data generation and visualization, specifically rare-event generation, through the use of GPU-accelerated simulation. Through this method, I also compare brute-force failure probability with the QMTM’s solution.

B. Minimum Volume Enclosing Ellipse

In order to generate bulk data, GPU parallelization had to be used. Then, each sample is mapped spatially to a thread, and steps are performed thread-wise, with all of the samples in shared memory. Stochastic noise is added to the the new state’s position and velocity. To avoid thread divergence, only a single step is run a time on the device.

To find the ellipses, bisection search combined with gradient descent is used. First, a circle is generated and is scaled to have a radius equal to the distance between the origin and the furthest state, and another circle is generated to have a radius equal to the distance between the origin and the closest non-zero state. The major and minor axes of the ellipses are then linearly interpolated (non-independently) to generate several intermediate ellipses. The two ellipses containing the fraction of points closest to P , one above and one below, are chosen, and the process is repeated until a given threshold is met. One challenge I encountered in this interpolation is that the area of an ellipse scales with the multiplication of the axes, meaning a linear interpolation of the axes results in a quadratic difference in area. However, although this slightly decreases the efficiency of the algorithm, it is not incorrect, and from testing, the excess time incurred is several orders of magnitude lower than the time taken for operations on the GPU, so I did not change this.

Without gradient descent, bisection search will find the enclosing ellipse that is a scaled version of the starting ellipse, but would likely not find the minimum volume enclosing ellipse. Running gradient descent with bisection search allows for independent change in the ratio of lengths of the axes and the angle of the ellipse and can converge to a minimum volume enclosing ellipse. These two algorithms must be used together to function as intended. Without bisection search, an ellipse after a gradient step would likely no longer contain the desired fraction of samples. In order to allow interactions between these algorithms, bisection search is modified to accept starting ellipses other than circles.

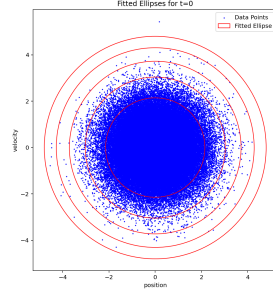
The gradient descent algorithm is initialized with a bisection search on a unit circle. Gradients are calculated by perturbing each ellipse parameter independently, running bisection search to return to containing 90% of points, and finding the derivative of the area with respect to the parameters. A step is taken down the slope of the gradient, and bisection search is applied one last time. This is repeated until convergence. Since bisection search

is applied throughout, this algorithm is extremely sensitive to small changes in parameters. Thus, another challenge I faced was overshooting in the gradient. This was partly due to the step size simply being too large and overshooting, but a lurking problem was that of the bisection search often needing to compensate for large changes in parameters by hugely increasing the area of the ellipse. This in turn causes large gradients, and even larger steps. Lowering the step size would have little effect and would result in large increases in computation time. It is θ , the angle of the ellipse, that bisection search is most sensitive to. For example, if the samples are spread across the x-axis and the ellipse is long and thin along the x-axis, a small change in θ , even as low as 5° , could result in a significant increase in size to encompass the required fraction of samples. A decreasing learning rate for all three parameters fixed this issue. The perturbation size and initial learning rate were chosen by a 20×20 grid search over perturbations in $[10^{-4}, 10^{-2}]$ and learning rates in $[10^{-3}, 10^{-1}]$, keeping the pair that produced the smallest final ellipse area. This gave a perturbation of 9.48×10^{-3} and an initial learning rate $\eta_0 = 4.27 \times 10^{-2}$. The learning rate decays as $\eta_k = \eta_0 / \sqrt{k}$, and updates to θ use $\eta_k / 50$.

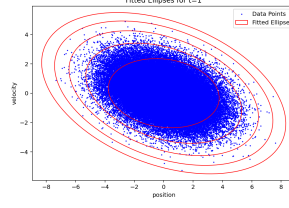
Finally, the combined algorithm of gradient descent and bisection search can be applied to finding multiple regions of $P\%$. A simple implementation would be to first generate some large quantity of samples, find the minimum volume enclosing ellipse of $P\%$, and repeat for the samples outside of this ellipse. However, this is quickly limited by the computational cost of running the bisection/gradient algorithm on so many samples. Although this could be alleviated by finding the MVEE on a subset of the samples, another limit is imposed by the available shared memory on GPUs during the data generation phase and step phases. For example, if 10, 90% are desired, and the minimum number of samples per region required to achieve meaningful results is 10^6 , then one would need to generate and store $\frac{10^6}{0.1^{10}-1} = 1e15$ samples, or one quadrillion samples. This is clearly infeasible, or at least undesirable. Instead, on-the-fly filtering is performed. A batch of samples is generated, where only the samples that are outside of the most recent region are accepted and written to memory. A scan is performed to find the number of accepted points. This is repeated until there are enough samples to get meaningful results. The iteration and kernel launch is done off-device to avoid thread divergence (i.e., only a single batch of points is generated at once). This can quickly and efficiently generate several million samples for very rare regions.

C. Results

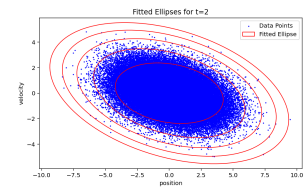
Figure 1 shows scatter plots across six time steps with five fitted ellipses each. A small percentage out of the original batches of samples was randomly selected for these plots. We observe that the initial distributions of states is Gaussian, consistent with our initial



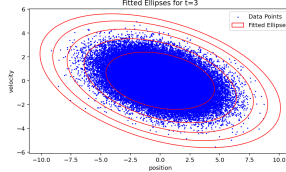
(a) fitted ellipses at $t=0$



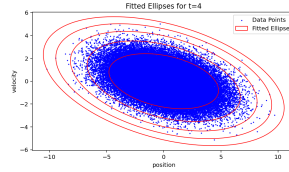
(b) fitted ellipses at $t=1$



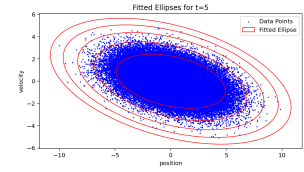
(c) fitted ellipses at $t=2$



(d) fitted ellipses at $t=3$



(e) fitted ellipses at $t=4$



(f) fitted ellipses at $t=5$

Figure 1: Shape of regions over time

sample generation method. The samples that have the same sign for their position and velocity have their velocity reduced, resulting in significant skew in the following time steps. Our ellipses capture this distribution, angling with the skew of the data. We observe that the further regions decrease in growth, likely due to the bulk of the data being near the center. A significant bit of intuition gained from these plots is that the $P\%$ region definition effectively captures the stochastic noise, being skewed probability levels. This simplicity does not typically exist in complex controllers, and warrants a more interesting exploration of a more complex non-linear model. For code documentation, see appendix A. To view the implementation, see the `toy_controller/in`.

IV. The Cart-Pole

A. Introduction

The Cart-Pole model, as introduced by [7], is a learned linear state-feedback controller, $F = \text{clip}(w^\top s, -30, 30)$ with $w = (0.6234, 1.8060, 34.6404, 11.8123)$, trained jointly with a neural Lyapunov function (a ReLU network with two hidden layers of 32 units) that balances an inverted pendulum on a cart. Bounds are set on the rail length and pendulum angle to define non-failure states. The Cart-Pole can be described with the following equation:

$$\begin{aligned}\ddot{x} &= \frac{F + m_p \sin \theta (l\dot{\theta}^2 - g \cos \theta)}{m_c + m_p \sin^2 \theta} \\ \ddot{\theta} &= \frac{-F \cos \theta - m_p l \dot{\theta}^2 \sin \theta \cos \theta + (m_c + m_p) g \sin \theta}{l (m_c + m_p \sin^2 \theta)} \\ s_{t+1} &= s_t + \Delta t (\dot{x}, \ddot{x}, \dot{\theta}, \ddot{\theta})\end{aligned}$$

Non-independent stochastic disturbance is then added to the output of the controller in order to emulate real-life situations. In order to build a QMTM for this model, good state partitions are required. However, P% ellipses become less useful with the introduction of non-linearity and rectangular bounds, as they do not capture the semantic differences between pockets of data. As previously mentioned, [7] present a learned Lyapunov function to solve for a measure of stability. A Lyapunov function is a strictly decreasing function that is bounded below by zero. Intuitively, it calculates the “energy” of a system, where a lower energy indicates a state closer to the nominal state, or equilibrium. In other words, a state with a given Lyapunov function will, without outside intervention, will forever remain in the subset of states with that Lyapunov function. However, equal Lyapunov functions do not necessarily indicate equal safety, as they do not account for future action. In addition, the Cart-Pole may approach constraint boundaries on its way to its stable destination, which can happen more than once. Clearly, the Lyapunov function is useful, but not in isolation. A cross of the Lyapunov with another measure of safety could provide good partitions. Consider these two measures L and r , where L is our Lyapunov function, and r is some other measure which captures future action.

A straightforward proposal for r could be the minimum future distance to failure in a model without stochastic disturbance. Computing this measure is relatively simple, but it turned out not to be useful for verifying failure probabilities for two, related reasons. A key observation is that this function tends to have extended plateaus where the trajectory is approaching the minimum distance to failure. Once it passes that point, the value of r climbs until it reaches value for the next minimum. This measure does not capture how a good controller actively corrects for disturbances that would push a trajectory towards failure.

Thus, stochastic disturbances create a “random walk” in the value of r , eventually leading to an unacceptable probability of failure. Furthermore, the QMTM formulation partitions the state space based on the value of r . Consider a region bounded between $r_{lo,1}$ and $r_{hi,1}$. By the QMTM model, a trajectory in this region could be **anywhere** in the region, including at $r_{lo,1}$. As noted above, after one time-step without the stochastic disturbance, the value of the r function in many cases will still be $r_{lo,1}$. When we add the disturbance, there is a large probability for a transition to the region with the next smaller range of values for r . This “wrapping effect”, causes the analysis to massively over approximate the failure probability.

B. Good Partitions

It is clear that we must find better partitions through a more thorough analysis of the system. Longer term, we hope that finding such partitions can be automated, quite possibly by using learning methods similar to the ones used by [7] for synthesizing a controller and Lyapunov function for the cart-pole. Systems in use in the world are typically far more complex than the Cart-Pole, and a deep exploration of the system to find useful partitions will not always be feasible, nor would it be exciting. A clear path forward with the Cart-Pole would be a decomposition based on the Eigenvalues and Eigenvectors of the Jacobian of the system dynamics. The Jacobian is the derivative of step function, $f(X)$, with respect to the components of the state. It is represented as an $n \times n$ matrix, where n is the dimension of the state. It is a first-order (Taylor) linear approximation of the step function around a given state. This means that entries represent how the step function’s output would be perturbed with small changes in the state. Its true use in this case comes from its eigenvalues and eigenvectors. Its eigenvalues give clear information about the stability of the system at a given state. Negative real-parts of the eigenvalues translate to an exponential convergence to the nominal state. Positive real-parts of the eigenvalues translate to divergence. Since the Jacobian is a linear approximation, these values are subject to sudden change across time steps, which means that eigenvalues indicating divergence for a state do not strictly imply that the state will diverge (and fail). Larger magnitudes directly imply faster convergence/divergence, but also imply more confidence in the posed outcome. Eigenvalues may have imaginary parts, and if they do, they always come in complex conjugate pairs. If there is a complex conjugate pair with negative real parts, this indicates that the spiral towards equilibrium, as opposed to converging directly towards it. Similarly, for a complex conjugate pair with positive real parts, this indicates a spiral away from equilibrium. When the real parts of eigenvalues are evaluated over time, they can provide current and near-future information about a state that can be used for partitioning.

This problem becomes more complex when constraints are placed on the maximum control the model can have on the system. When the model outputs a very high force to an unrealistic extent, it is not illogical to impose some upper bound on the force the model can exert. When the model reaches this upper bound, it is in a saturated state. The system acts inherently differently in an unsaturated state, notably there are less effects across components of the state with the output state, meaning the Jacobians are simpler (and different) than the unsaturated Jacobians. There is then a division in states before we even evaluate the eigenvalues: Unsaturated and saturated.

C. Results

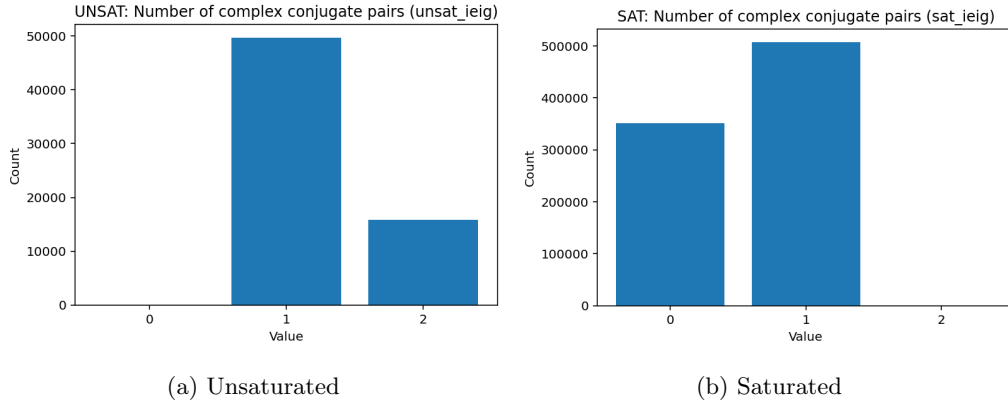


Figure 2: Number of complex conjugate pairs in saturated and unsaturated samples

We have established that positive eigenvalues indicate instability, whereas negative eigenvalues indicate stability. There is also the issue of saturation. To gain some intuition into the possible configurations of the eigenvalues, bulk simulations were run. Figure 2 show the number of complex conjugate pairs in the saturated and unsaturated states over a grid of several hundred thousand samples. We can see that unsaturated samples have 0-2 positive real parts, meaning at least 2 out of 4 eigenvalues are negative, with a strong skew towards 0 positive real parts. Saturated samples also have 0-2 positive real parts, with a strong skew towards 1-2 positive real parts. We also observe that there is always at least one complex conjugate pair for unsaturated states, and never more than one for saturated states. It would also be useful to find the spreads and separations of the real parts within sets of eigenvalues to identify the differing rates of convergence, where the spread is the largest-magnitude real part divided by the smallest-magnitude real part, and the separation is the second smallest divided by the smallest. To avoid division by zero errors, 3 shows the inverse of these values for unsaturated states. A smaller inverse of spread and separation

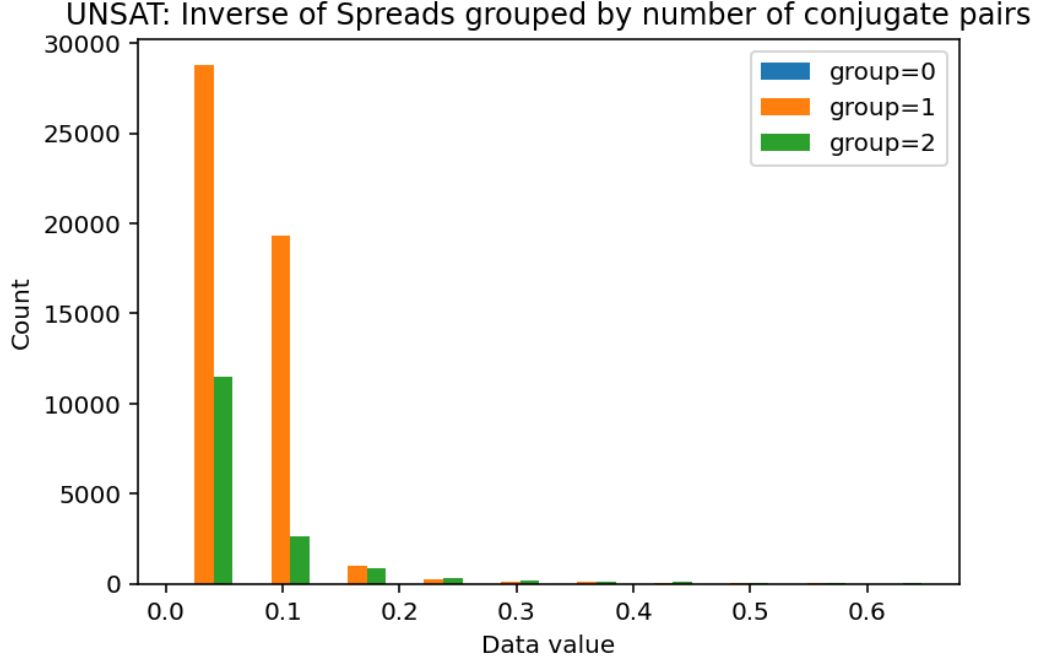


Figure 3: Enter Caption

corresponds to a larger difference in rates of convergence.

We can then discretize the state into several modes, depending on the eigenvalues and the state itself: A FAIL state indicates that the state violates a safety constraint. All states in remaining modes satisfy the safety constraint. A SAT state indicates that the value is saturated at its maximum or minimum value. A POS state indicates that the Jacobian has at least one eigenvalue with a positive real part. RNEG indicates that all eigenvalues of the Jacobian are real and negative. J1 and J2 indicate that there are one and two complex-conjugate pairs of eigenvalues, respectively. Finding the eigenvalues of the Jacobian of the setpoint $(0, 0, 0, 0)$ shows us that our setpoint is in J1. If we were to plot the trajectory a state on the imaginary/real plane that, for example, begins in J2 and ends at equilibrium in J1, we would witness one of the complex conjugate pairs circling in towards the real axis, and once reached, splitting off towards the equilibrium real values. The other complex conjugate pair would circle towards the equilibrium's complex conjugate values. Figure 4 shows the possible state transitions using the above partitions. Since J1 can transition to FAIL, being in J1 does not guarantee convergence. An avenue for future work would be to find the boundaries of these regions using a flood-fill algorithm on the GPU.

Transition	Count	Min Steps	Max Steps	Avg Steps	Std Dev	Median Steps
SAT $\rightarrow$ FAIL	8818	1	9	1.65	1.03	1.0
SAT $\rightarrow$ J1	218	1	9	3.17	1.64	3.0
SAT $\rightarrow$ POS	206	1	8	2.92	1.99	3.0
SAT $\rightarrow$ J2	100	1	4	2.38	1.06	2.5
POS $\rightarrow$ J1	164	1	12	2.27	2.33	1.0
POS $\rightarrow$ J2	182	1	3	1.49	0.65	1.0
POS $\rightarrow$ FAIL	210	1	7	2.31	1.40	2.0
POS $\rightarrow$ SAT	2	1	1	1.00	0.00	1.0
J2 $\rightarrow$ J1	428	1	5	2.00	0.97	2.0
J2 $\rightarrow$ FAIL	66	1	3	1.42	0.60	1.0
J1 $\rightarrow$ FAIL	638	1	18	5.26	4.10	4.0
J1 $\rightarrow$ POS	130	1	10	4.35	2.36	4.0
J1 $\rightarrow$ J2	46	1	2	1.39	0.49	1.0

Figure 4: Possible transitions between states, derived from Monte-Carlo simulation

D. Implementation

The Python implementation of this method uses PyTorch to store and manipulate data in-simulation. PyTorch’s auto-gradient is used to automatically differentiate the states. Specifically, the changes made over a step are tracked and automatically differentiated. PyTorch has a Jacobian solver that uses the auto-diff states. The eigenvalues and vectors of this Jacobian are found using NumPy’s linear algebra module. NumPy is also used for data processing. Matplotlib is used for data visualization.

E. Results

Through the use of auto-differentiation, we were able to generate first-order linearizations of our step function, and from this, we generated the Jacobian. The Jacobian is a measure of how perturbations in the components of the state would affect the output step function $f(x)$. If we find the eigenvalues and eigenvectors of the Jacobian, we can split the effect of perturbations into individual modes. The eigenvectors represent a change of base for the state space that allow us to identify stability in each independent direction. The eigenvalues can then be read to gain insights into what mode the state is headed to: stability (negative real-part eigenvalue), instability (positive real-part eigenvalue), or marginality (zero real-part eigenvalue), as well as whether the state will oscillate as it is in one of these modes (complex conjugate pair). This divide in modes is a promising avenue for region definition. However this method of partitioning the state space is not yet fully defined, so plots and graphs help gain intuition and insights. Some key insights include the discovery of the set-point’s Jacobian’s eigenvalues and its related state J1, the distribution and path of eigenvalues in J1, and the identification of and transition possibilities between regions. An avenue of further work could be in identifying possible saddle points, as it was found that

there were some states that remained in POS for an abnormal duration. The setpoint's Jacobian lies in J1, and J1 eigenvalues follow a consistent path: the complex-conjugate pair spirals toward the real axis while the two real eigenvalues stay separated (one near -6 to -7 , one between -2 and 0). Simulating 10,000 grid-sampled initial states showed that 9,340 began saturated, and 9,732 ended in FAIL. Of the 272 states that began in J1, 268 ended in J1, confirming that J1 is strongly but not perfectly stable. Saturation, rather than the eigenvalue structure alone, is the dominant predictor of failure. See Appendix B for brief documentation on the implementation, and the `cartpole/in`.

V. Conclusions and Future Work

A. Contributions

This thesis makes several key contributions to the challenge of verifying extremely low failure probabilities in nonlinear control systems. Conceptually, it bridges the gap between formal verification and simulation by using large-scale Monte Carlo data to inform and refine the Quasi-Markov Chain (QMC) framework for safety analysis. In particular, I demonstrate how high-volume GPU-based simulations can illuminate the state-space structure, providing intuition for effective partitioning of continuous states into discrete regions. This data-driven visualization serves as a debugging tool for the human engineer to adjust proposed partitions. For example, revealing when a naive partition misses a skewed concentration of states or lumps together dynamics that should be distinguished. By gaining insight into the distribution of states (e.g., identifying pockets of rare but important trajectories) and the controller’s behavior in those regions, this work demystifies the art of partition selection. It also provides an empirical validation of the QMC approach: by comparing simulated failure frequencies with the QMC’s upper-bound estimates, we can quantify how conservative those bounds are and ensure they tighten around actual probabilities.

Technically, the thesis introduces a novel pipeline that combines GPU-accelerated sampling with advanced optimization for region shaping. A custom CUDA-based simulation framework was developed to generate millions of system trajectories in parallel, making it feasible to observe failure events with probabilities on the order of 10^{-7} to 10^{-8} . This massive data generation is paired with an efficient algorithm to compute Minimum Volume Enclosing Ellipses (MVEE) for capturing a chosen percentage of sample points. By applying a bisection search over candidate ellipsoids and refining their axes via gradient descent, the method finds tight-fitting ellipsoidal regions that enclose, say, 90% of the data, improving upon grid-based partitions. These ellipsoidal regions better capture the true shape of state distributions (including skew and orientation) at each time step, leading to tighter upper bounds on transition probabilities between regions. In search of better region definitions for nonlinear controllers, I leveraged Jacobian-based heuristics and Lyapunov function measures to guide region definition. Automatic differentiation was used to linearize the controller dynamics and compute eigenvalues of the Jacobian across the state space. This allowed us to identify different dynamic regimes (such as rapidly convergent vs. oscillatory modes) and ensure the partitioning scheme accounts for them. The resulting framework is a combination of simulation, geometric optimization, and system-theoretic insight.

Together, these contributions extend Reiher’s QMC verification method with practical, sampling-based tools that make it more usable and informative for complex systems. In

summary, combining GPU-powered Monte Carlo analysis with informed partitioning is a promising way to improve our ability to quantify and upper-bound the risk of rare controller failures.

B. What I Learned

Learning about and participating in the field of GPU-accelerated/parallel programming has greatly expanded my repertoire of technical skills and knowledge. The power that lies in a parallelization task, and finding parallelisms where they intuitively shouldn't be, like scan operations, has brought me joy. One of the most difficult things in this thesis was diving head-first into several fields that I had very little prior experience in, such as GPU-accelerated programming and ODE applied to linear algebra. I felt somewhat like I was constantly running to keep up, and balancing this with a full course load pushed me to my limits.

This head-first dive resulted in a lot of first-time mistakes, such as writing in `CUDA C` instead of `CUDA C++`, which greatly slowed down my progress and frustrated me quite a bit with the ridiculous amount of boilerplate code and hidden bugs. However, from these frustrating experiences, I've learned to prepare, research, and then dive head first (possibly with my arms out, this time).

My favorite moments were my regular meetings with Professor Greenstreet. Through our discussions, I listened to decades of knowledge distilled into a few sentences or a drawing or two on a whiteboard. I learned that research is not an elitist community, but rather a space where people come together to think as hard as they can, and fail a lot. I think this humanization of research is what made me want to pursue research further. I am chasing those moments where I was able to speak on the same level as my supervisor.

Future Work

While this thesis developed a viable approach for tighter probabilistic safety bounds, there remain many opportunities to extend and refine the work. One important direction is to apply the methodology to more complex or hybrid dynamics, such as the Van der Pol oscillator with state-dependent switching behavior. A system like the Van der Pol, which exhibits oscillatory limit-cycle dynamics and can switch between different modes of behavior, would be a natural next step for generalizing a partitioning method. Adapting our approach to such a scenario may require introducing new partition definitions or time-varying region shapes, but it would demonstrate the framework's generality. Success on a challenging nonlinear controller would show that the combined QMC and simulation technique can handle the more complex dynamics found in real-world controllers.

On the implementation side, there are clear avenues for optimization that could improve performance and scalability. For example, porting the optimization routines and inflation sampling fully to the GPU (CUDA) is a natural next step. This improvement would be especially valuable when dealing with computation-heavy higher-dimensional systems, as it would keep the entire verification pipeline parallelized and efficient. Overall, continued profiling and low-level optimization of the GPU code (e.g. minimizing memory overhead, maximizing coalesced access) could allow the approach to scale much easier.

Another avenue is to use GPU-accelerated code to expose and test boundary points or clusters of rare events on custom controllers to test Reiher’s method. One could incorporate rare-event sampling techniques to find these clusters. Analyzing these clusters could also suggest refinements in the partitioning method, such as more precise definition for failure regions.

Further, a learned partitioner could be of great use, especially since the purpose of this tool is to be used generally. Although the use of a Lyapunov function is not currently required, it greatly increases the precision of the upper-bound failure probabilities, and is therefore necessary to achieve any reasonable result. Using learned-Lyapunov functions or learning some measure of stability would render this method completely generalizable. Theoretically, one could also use a GPU-based flood-fill technique to discover region boundaries given some definition of mode, by finding contiguous areas where the system’s linearized behaviour stays within a certain category. There are several avenues for partitioning refinement, but the strength in Reiher’s method is that it does not absolutely need them.

A: Toy Controller

This code base is located in `/toy_controller` of the repository (<https://github.com/superliminal/honours-thesis>). It consists of the CUDA C region finder and a short Python plotting script that reads its output.

A.1: CUDA C

The program is built with `make` (which requires `nvcc`) and was run on the UBC Computer Science GPU servers (see this UBC guide). `run_program.sh` submits it as a SLURM job. The output directories `logs/` and `plot_info/` must exist before running, since the logger and the plotting output write into them.

Command line options are `-n` (number of samples), `-t` (time horizon), `-kp` (proportional gain), `-kd` (derivative gain), `-nom` (nominal state) and `-log` (logging level). The defaults set in `main.cu` are a horizon of 5 time steps, $K_p = -0.25$, $K_d = -1.5$, a nominal state of $(0,0)$ and logging level 1. The number of samples is currently fixed at 1,000,000 inside `find_regions`, so `-n` has no effect. With the default settings the program takes about 10 minutes.

Most of the running time is spent on the GPU. The bisection search and gradient descent loops are driven from the host (CPU), but the distance and point-counting kernels they call run on the GPU, and filtering (finding samples in rare regions) dominates the running time. The host-side loops are therefore cheap to extend, for example by increasing the maximum number of bisection iterations. Ellipses are stored as a semi-major axis a , a semi-minor axis b , and an orientation θ in radians.

`main.cu`

```
1 int main(int argc, char* argv[])
```

Parses the command line options, initializes the logger and the simulation metadata, and runs the region finder (see `regions.h` below) for 5 regions per time step. Logs the time taken to find the regions.

`logger.h`

A module for controlled, leveled logging. There are four log levels: `LOG_ERROR` (0, sparsest), `LOG_BASIC` (1), `LOG_ADVANCED` (2) and `LOG_DEBUG` (3, densest). Messages are written to `stdout` and to a time-stamped file in `logs/`. The module also saves states and ellipses to

plot_info/, which the Python script uses for plotting. At most the first 100,000 states are saved per time step, to keep the files small.

simulation.h

```

1 typedef struct Sim_Metadata;
2 typedef struct Samples;
3 Samples* samples_init_random(int n);
4 void samples_free(Samples* samples);
5 void step(Samples *samples, Sim_Metadata *meta, Logger *logger);

```

Sim_Metadata holds the number of samples, the time horizon, the current time step, the controller gains and the nominal state. **Samples** holds matching host and device arrays of states, each a `float2` of position and velocity. `samples_init_random` allocates `n` samples and draws their initial states from a standard normal distribution on the GPU. `step` advances every sample by one time step in parallel through the `update_positions` kernel, with one thread per sample. Each thread applies the PD update $x \leftarrow x + \dot{x}$, $\dot{x} \leftarrow \dot{x} + K_p x + K_d \dot{x}$ (a time step of 1), then adds independent standard normal noise to both components. The updated states are copied back to the host.

bisection.h

```

1 typedef struct Bisection;
2 Bisection* bisection_init(Samples* samples, Logger *logger,
3     int num_ellipses, Ellipse initial_ellipse);
4 void bisection_free(Bisection* bisection);

```

Struct definition, initialization and destruction, used internally for simpler data management. A **Bisection** struct contains the working ellipses, the number of them, the number of samples within each ellipse, the normalized distance from each sample to the ellipse, and the **low** and **high** bracketing ellipses. `bisection_init` scales the initial ellipse uniformly so that **low** passes through the sample closest to the origin and **high** through the farthest, so they contain approximately 0% and 100% of the samples respectively. If **low** already contains at least the target fraction, or **high** contains at most it, the function returns `NULL`. This edge case could likely be handled better. `bisection_init` should not be called outside of the **bisection** module; the single entry point is `bisection_search`.

```

1 #define EPSILON_BISECT 1e-15f
2 #define TARGET_PERCENTAGE 0.9f
3 #define MAX_ITERATIONS_BISECT 25
4 #define NUM_ELLIPSES 10

```

Constants. `EPSILON_BISECT` and `MAX_ITERATIONS_BISECT` are used for the termination conditions. `TARGET_PERCENTAGE` is the fraction of samples that should lie inside an ellipse. `NUM_ELLIPSES` is the number of ellipses generated at each iteration of bisection search.

```

1 void generate_ellipses(Bisection* bisection, Logger *logger);

```

Generates `NUM_ELLIPSES` ellipses between the `low` and `high` ellipses through linear interpolation of the semi-axes and the angle, and copies them to the device. Since the area scales with the product of the semi-axes, the difference in area between neighbouring interpolated ellipses is not constant. Interpolating on a logarithmic scale may speed up convergence.

```

1 int bisection_search(Samples* samples, Sim_Metadata* meta, Logger* logger, Ellipse* ellipse);

```

Initializes the bisection struct using `ellipse` as the starting shape, and later overwrites `ellipse` with the best ellipse found. Each iteration generates the interpolated ellipses and counts the samples inside each one on the GPU. It then selects the ellipse whose coverage is closest to the target from above as the new `high`, and the one closest from below as the new `low`. The search stops when either bracket is within $0.01/n$ of the target fraction, when the two brackets converge, or after `MAX_ITERATIONS_BISECT` iterations. Returns 0 if initialization failed and 1 otherwise.

gradient.h

The gradient descent module. It uses bisection search extensively.

```

1 #define PERTURBATION 9.478947e-3f
2 #define LEARNING_RATE 4.268421e-2f
3 #define EPSILON_GRAD 2e-4f
4 #define MAX_ITERATIONS_GRAD 100
5 #define MAX_ITERATIONS_SINCE_BEST 15
6 void hyperparameter_tuning(Samples *samples, Sim_Metadata *meta, Logger* logger);

```

Constants and the hyperparameter tuning function. `PERTURBATION` is the step used to estimate each partial derivative by finite differences, and `LEARNING_RATE` is the initial step size along the gradient. Both were chosen with `hyperparameter_tuning`, which runs gradient descent over a 20×20 grid of perturbations in $[10^{-4}, 10^{-2}]$ and learning rates in $[10^{-3}, 10^{-1}]$ and keeps the pair that produces the smallest ellipse. The same values are shared by all three ellipse parameters, except that the step for θ is divided by 50. `hyperparameter_tuning` must be invoked manually and is not called anywhere in the current code base.

```

1 typedef struct Gradient;
2 Gradient* gradient_init(float perturbation, float initial_learning_rate);
3 void gradient_free(Gradient* gradient);

```

The `Gradient` struct contains the current ellipse, a scratch ellipse for perturbations, an array for the gradient, and the hyperparameters (perturbation, initial learning rate, convergence threshold and maximum number of iterations). `gradient_init` allocates memory, sets the hyperparameters, and initializes the values. Like `bisection_init`, it is only used within the `gradient` module to reduce coupling between modules. The entry points to `gradient` are `gradient_descent` and `hyperparameter_tuning`.

```

1 float gradient_descent(Samples *samples, Sim_Metadata *meta, Logger *logger,
2 float perturbation, float initial_learning_rate, Ellipse* best_ellipse);

```

Gradient descent begins by running bisection search on a unit circle and records the resulting area as the best so far. Each iteration first sets the learning rate to $\alpha_k = \alpha_0 / \sqrt{k}$, where k is the iteration counter. It then estimates the partial derivatives of the ellipse's area A with respect to a , b and θ by finite differences: each parameter is perturbed, bisection search restores the target coverage, and the change in area is divided by the perturbation. If every partial derivative is below `EPSILON_GRAD`, the loop exits. Otherwise the ellipse is updated by

$$x \leftarrow x - \alpha_k \frac{\partial A}{\partial x} \quad \text{for } x \in \{a, b\}, \quad \theta \leftarrow \theta - \frac{\alpha_k}{50} \frac{\partial A}{\partial \theta}.$$

If the updated ellipse is invalid (for example, a negative semi-axis), the loop ends. Otherwise bisection search is run on the new ellipse, and if its area is the smallest so far it becomes the best ellipse. The loop also ends when the change in area between iterations is below the threshold, after `MAX_ITERATIONS_GRAD` iterations, or when more than `MAX_ITERATIONS_SINCE_BEST` iterations have passed without improvement (for example, due

to repeated overshooting). The best ellipse is written to `best_ellipse`, and its area is returned.

regions.h

The region-finding module. It uses gradient descent extensively.

```
1 typedef struct Iteration_Info;
2 Iteration_Info* iteration_init(int n);
3 void iteration_free(Iteration_Info* info);
```

`Iteration_Info` stores the working samples and ellipse for one time step of the algorithm. `iteration_init` allocates memory and generates `n` random samples.

```
1 void find_regions(Sim_Metadata *meta, Logger *logger, int num_regions);
```

Called by `main`. For every time step up to the time horizon, it generates 1,000,000 random samples, advances them to the current time step, and saves the states to `plot_info/`. It then runs gradient descent to find the first region and saves the resulting ellipse. For each further region, it replaces the samples with new ones that lie outside the most recent ellipse (see `generate_points` below), runs gradient descent on them, and saves the resulting ellipse. All ellipses for a time step are saved to the same file, and running the program again overwrites earlier files. Saving a fixed number of samples per region, rather than only the first samples of each time step, could give a better visualization of the outer regions.

utility.h

A general-purpose utility module, created to reduce code duplication. It contains the error-checking macro `cudaTry`, the `Ellipse` type, small kernels such as random number generation and array initialization, the kernel that counts samples inside ellipses, and the on-the-fly filtering used to sample rare regions:

```
1 void generate_points(Samples* samples, Ellipse* h_ellipse, int k,
2 float kp, float kd, Logger* logger);
```

Replaces the contents of `samples` with states that lie outside `h_ellipse` at time step `k`. Each launch of `generate_points_kernel` runs twice as many threads as samples are needed. Each thread draws a random initial state, simulates it for `k` steps with noise, and flags whether the

final state lies outside the ellipse. Within each block, a parallel prefix scan over the flags in shared memory gives every accepted state its position in the block's output, and one atomic addition per block reserves space in a global output buffer. The host relaunches the kernel until enough states have been accepted, then copies the first `n` of them into `samples`. Only accepted states are ever written to memory, so the outer regions can be sampled without storing the full set of simulated trajectories.

A.2: Python

`plotting_script/plot.py`

Reads `plot_info/states_t.txt` and `plot_info/ellipses_t.txt` for time steps 0 to 5 and saves a scatter plot of the states with the fitted ellipses overlaid to `plots/ellipses_t.png`. It uses paths relative to `plotting_script/`, so it should be run from that directory. These plots make up Figure 1.

B: Cart-Pole

B.1: Python

This code is located in `/cart_pole/python` of the repository. It consists of the following files, plus a Conda environment file (`environment.yml`) that includes the relevant libraries.

- `cp.py`: the main file, which classifies states into modes and records the transitions between them. Its functions are described below.
- `cp_eigdata.py`: computes the Jacobians of a grid of states, split into saturated and unsaturated states, and plots summary statistics of their eigenvalues, including the number of complex-conjugate pairs and the inverse spreads and separations of the real parts.
- `cp_eigsummary.py`: produces a table of summary statistics for the real parts of the eigenvalues of saturated and unsaturated states.
- `readpkl.py`: a small helper that loads the saved Jacobians (`cp_jacobians.pkl`) and prints how many saturated and unsaturated samples they contain.
- `cartpole.py` and the two `.pt` files: the controller and Lyapunov function learned by [7], with the accompanying code from that work.

For brevity, I will not explain the plotting functions, as these will likely be replaced in later revisions and do not contain core functionality.

```
1 def data_processing(load:bool = False, save:bool = False, filename = "storage_unit.pkl")
```

The main entry point of `cp.py`. Exactly one of `load` and `save` must be `True`, otherwise the function exits. With `load`, it reads a previously saved “storage unit” (a large dictionary) from `filename`. With `save`, it generates new data over a $10 \times 10 \times 10 \times 10$ grid of 10,000 states and saves it to `filename`; generating data takes significantly longer than loading it. In both cases it then plots the distribution of starting and ending modes, the transition statistics, and the eigenvalues of states that end in J1. As written, the file calls `data_processing(load=True)`.

```
1 def make_storage_unit(n_x = 5, n_xdot = 5, n_theta = 5, n_thetadot = 5)
```

The main data generation function. It generates a grid of states and their Jacobians, classifies each state into a mode (FAIL, SAT, POS, RNEG, J1 or J2), and sorts the states by mode. It then steps each state that has not failed forward up to 20 times, stopping early if it reaches FAIL, and records every state visited along with every change of mode as a transition. Finally, it places the starting and ending states and modes, the full trajectories and the transitions in the storage unit. The storage unit can be extended with additional data, but changes must be made in both `make_storage_unit` and `data_processing`.

```
1 def generate_J_and_states(n_x=5, n_xdot=5, n_theta=5, n_thetadot=5)
```

Generates a grid of states (using `generate_cp_states` below, with the given number of points along each axis) and the Jacobian at each state, and returns them as a tuple.

```
1 def generate_cp_states(n_x=5, n_xdot=5, n_theta=5, n_thetadot=5)
```

Generates a grid of states. It spaces points evenly across the position and angle constraints from `cart_pole_constraints()` and across the linear and angular velocity bounds from `implied_cp_constraints()`, then takes every combination. It returns a tensor in which each row is a state and each column is one component of the state.

```
1 def cp_step(state: torch.tensor)
2 def ode_step(state: torch.tensor)
```

`cp_step` returns the next state (along with the force applied), whereas `ode_step` returns the change in state over one time step. `cp_step` is used in the saturation check and to step states forward in `make_storage_unit`. `ode_step` is used to calculate the Jacobian.

```
1 def get_Jacobian(state: torch.tensor)
```

Returns the Jacobian of `ode_step` at the given state using `torch.autograd.functional.jacobian`, which differentiates the function with respect to its input directly.

References

- [1] Gul Agha and Karl Palmskog. “A survey of statistical model checking”. In: *ACM Transactions on Modeling and Computer Simulation (TOMACS)* 28.1 (2018), pp. 1–39.
- [2] Mansur Arief et al. “Certifiable deep importance sampling for rare-event simulation of black-box systems”. In: *arXiv preprint arXiv:2111.02204* (2021).
- [3] Mansur Arief et al. “Certifiable Evaluation for Autonomous Vehicle Perception Systems using Deep Importance Sampling (Deep IS)”. In: *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*. 2022, pp. 1736–1742. DOI: 10.1109/ITSC55140.2022.9922202.
- [4] Siu-Kui Au and James L. Beck. “Estimation of small failure probabilities in high dimensions by subset simulation”. In: *Probabilistic Engineering Mechanics* 16.4 (2001), pp. 263–277. ISSN: 0266-8920. DOI: [https://doi.org/10.1016/S0266-8920\(01\)00019-4](https://doi.org/10.1016/S0266-8920(01)00019-4). URL: <https://www.sciencedirect.com/science/article/pii/S0266892001000194>.
- [5] “Building the Stochastic Matrix”. In: *Markov Chains*. John Wiley Sons, Ltd, 2017. Chap. 3, pp. 25–35. ISBN: 9781119387596. DOI: <https://doi.org/10.1002/9781119387596.ch3>. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/9781119387596.ch3>. URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/9781119387596.ch3>.
- [6] M. Chen, S. Herbert, and C. J. Tomlin. “Fast reachable set approximations via state decoupling disturbances”. In: *2016 IEEE 55th Conference on Decision and Control (CDC)*. Dec. 2016, pp. 191–196. DOI: 10.1109/CDC.2016.7798268.
- [7] Charles Dawson, Sicun Gao, and Chuchu Fan. “Safe Control With Learned Certificates: A Survey of Neural Lyapunov, Barrier, and Contraction Methods for Robotics and Control”. In: *Trans. Rob.* 39.3 (June 2023), pp. 1749–1767. ISSN: 1552-3098. DOI: 10.1109/TR0.2022.3232542. URL: <https://doi.org/10.1109/TR0.2022.3232542>.
- [8] Jinchao Huang and Sibor Wang. *Subset Sampling and Its Extensions*. 2023. arXiv: 2307.11585 [cs.DS]. URL: <https://arxiv.org/abs/2307.11585>.
- [9] Jeffrey Humpherys, Preston Redd, and Jeremy West. “A Fresh Look at the Kalman Filter”. In: *SIAM Review* 54.4 (2012), pp. 801–823. DOI: 10.1137/100799666. eprint: <https://doi.org/10.1137/100799666>. URL: <https://doi.org/10.1137/100799666>.
- [10] I.M. Mitchell, A.M. Bayen, and C.J. Tomlin. “A time-dependent Hamilton-Jacobi formulation of reachable sets for continuous dynamic games”. In: *IEEE Transactions on Automatic Control* 50.7 (2005), pp. 947–957. DOI: 10.1109/TAC.2005.851439.

- [11] Justin Reiher. “Efficiently computing upper bound failure probabilities of controlled probabilistic systems”. PhD Thesis Proposal. University of British Columbia, July 2024.
- [12] Guy Scher, Sadra Sadraddini, and Hadas Kress-Gazit. “Probabilistic Rare-Event Verification for Temporal Logic Robot Tasks”. In: *2023 IEEE International Conference on Robotics and Automation (ICRA)*. 2023, pp. 12409–12415. DOI: 10.1109/ICRA48891.2023.10161083.
- [13] Guy Scher et al. “Elliptical Slice Sampling for Probabilistic Verification of Stochastic Systems with Signal Temporal Logic Specifications”. In: *Proceedings of the 25th ACM International Conference on Hybrid Systems: Computation and Control*. HSCC ’22. Milan, Italy: Association for Computing Machinery, 2022. ISBN: 9781450391962. DOI: 10.1145/3501710.3519506. URL: <https://doi.org/10.1145/3501710.3519506>.
- [14] Adam Thorpe and Meeko Oishi. “SOCKS: A Stochastic Optimal Control and Reachability Toolbox Using Kernel Methods”. In: *Proceedings of the 25th ACM International Conference on Hybrid Systems: Computation and Control*. HSCC ’22. Milan, Italy: Association for Computing Machinery, 2022. ISBN: 9781450391962. DOI: 10.1145/3501710.3519525. URL: <https://doi.org/10.1145/3501710.3519525>.
- [15] E.A. Wan and R. Van Der Merwe. “The unscented Kalman filter for nonlinear estimation”. In: *Proceedings of the IEEE 2000 Adaptive Systems for Signal Processing, Communications, and Control Symposium (Cat. No.00EX373)*. 2000, pp. 153–158. DOI: 10.1109/ASSPCC.2000.882463.
- [16] WEofferwhatYOUwant. *Plush Toy Dog That Obey's Your Voice Commands. 12 Tricks Walks Barks Sings and Dances. Teddy The Labradoodle for Ages 4 to 8 Years Old and Above*. Available at: <https://www.amazon.com/WEofferwhatYOUwant-Commands-Tricks-Dances-Labradoodle/dp/B0B86Y1R19>. 2024.